PRD – Simulafoot

Version : 1.0
Date : 10 octobre 2026
Statut : Draft
Auteur : Assistant, basé sur les réponses de l'utilisateur
Destinataire : Porteur du projet Simulafoot

Table des matières

1. Vision et objectifs
2. Utilisateurs cibles et cas d'usage
3. Périmètre fonctionnel
4. Exigences fonctionnelles
5. Narration et NLG
6. UI/UX
7. Architecture technique
8. Modèle de données
9. Roadmap et priorités
10. Métriques de succès
11. Risques et contraintes
12. Décisions à prendre
13. Annexes

1. Vision et objectifs

1.1. Vision

Simulafoot est un **moteur de simulation de matchs de football** le plus réaliste possible, en français, qui génère des **matchs narratifs uniques** à partir d'une base de 7 564 joueurs enrichie de 47 attributs FM26. Le projet est personnel, mais destiné à être partagé avec des proches.

1.2. Problème résolu

Il n'existe pas d'outil de simulation de football réaliste, narratif et en français, accessible à un utilisateur occasionnel. Les jeux existants sont souvent en anglais, complexes, ou peu narratifs.

1.3. Promesse utilisateur

> « Vis des matchs de football réalistes, racontés comme un commentateur enthousiaste, avec des joueurs qui ont un vrai profil. »

1.4. Objectifs mesurables

- Simuler un match en moins de 30 secondes.
- Générer 10 à 20 temps forts par match, sans répétition perceptible.
- Produire un score et des statistiques cohérents avec les notes FM des joueurs.
- Offrir une expérience fluide sur web, desktop et mobile.
- Atteindre un niveau de finition « produit fini » à terme.

1.5. Modèle économique

Aucun. Projet personnel, partage gratuit avec des proches.

---

2. Utilisateurs cibles et cas d'usage

2.1. Utilisateurs

Profil	Description	Fréquence
Porteur du projet	Développeur et utilisateur principal	Quotidienne
Amis / proches	Joueurs occasionnels, connaisseurs de football	Occasionnelle

2.2. Cas d'usage principaux

- Simuler un match amical ou de championnat.
- Simuler une saison complète.
- Consulter les statistiques et temps forts d'un match.

- Comparer des joueurs (à terme).
- Partager un résumé de match avec un proche.

2.3. Cas d'usage secondaires

- Consulter la fiche d'un joueur.
- Explorer les classements.
- Suivre l'évolution d'une équipe sur une saison.

---

3. Périmètre fonctionnel

3.1. Must have (V1)

- Simulation de saison complète.
- Moteur de match action par action.
- Génération de temps forts textuels (10 à 20 par match).
- Score, statistiques et visualisation simple.
- Interface web, desktop et mobile.
- Base de données joueurs avec attributs FM26.
- Calcul de la note FM.

3.2. Nice to have

- Statistiques avancées (xG, heatmaps, percentiles).
- Comparateur de joueurs.
- Fiche joueur détaillée.
- Export de données.
- Mode carrière.

3.3. Hors périmètre

- Jeu en ligne multijoueur.
- Contrôle direct d'une équipe ou d'un joueur.
- 3D ou animation temps réel.
- Multilingue (français uniquement).

3.4. Priorités

1. Simulation réaliste.
2. Ergonomie et expérience utilisateur.
3. Narration riche et variée.

---

4. Exigences fonctionnelles

4.1. Moteur de simulation

Exigence	Détail
Type	Probabiliste, action par action
Granularité	Chaque action est simulée individuellement
Entrées	Notes FM, attributs, tactiques, compositions, forme, moral
Sorties	Score, statistiques, temps forts textuels, visualisation
Gestion	Tactiques, compositions, remplacements, blessures, cartons, forme, moral
Vitesse	Mode rapide (quelques secondes) et mode détaillé (plusieurs minutes)
Calibration	Basée sur les notes FM et les attributs individuels
Périmètre	Matchs amicaux, championnats, compétitions, transferts (à terme)

4.2. Données

Exigence	Détail
Source de vérité	Excel (`data/joueurs.xlsx`) + SQLite (à terme)
Mise à jour	Oui, à chaque patch FM
Attributs	47 attributs FM26, postes FM, niveau FM
Potentiel / attributs cachés	Non collectés
Colonnes ajoutées	Note FM poste, Note FM rôle, Rôle FM retenu, Note FM

4.3. Interface

Exigence	Détail
Support	Web, desktop, mobile
Écran prioritaire	Simulation de saison
Niveau de finition	Produit fini
Design	Jeu vidéo
Logo	Existant
Accessibilité	À définir (proposition : niveau AA)

---

5. Narration et NLG

5.1. Ton

Enthousiaste, dynamique, non neutre. Varie selon l'importance du match, la minute, le score, le joueur et l'équipe.

5.2. Format

- Longueur d'un temps fort : 2 à 3 phrases.
- Nombre de temps forts par match : 10 à 20, voire plus.
- Langue : français.

5.3. Anti-répétition

- Mémoire par match.
- Cooldown par phrase.
- Pénalité de récence.
- Journalisation des usages.

5.4. Profil joueur

Le profil influence :

- le choix des phrases,
- les slots dynamiques,
- le ton.

5.5. Événements couverts

Tout événement marquant : but, tir, arrêt, passe décisive, tacle, carton, blessure, remplacement, occasion manquée, etc.

5.6. Architecture NLG

- Actions complètes pré-écrites (approche 3).
- Matrice de compatibilité pour les compositions (approche 1).
- Conditions de profil (attributs, poste, rôle, âge, taille, pied, note, stats).
- Slots dynamiques.
- Post-traitement linguistique.

---

6. UI/UX

6.1. Principes

- Interface de jeu vidéo.
- Navigation fluide.
- Hiérarchie claire.
- Données denses mais lisibles.
- Feedback immédiat.

6.2. Écrans prioritaires

1. Simulation de saison.
2. Simulation de match.
3. Classement.
4. Fiche joueur.
5. Comparateur (nice to have).

6.3. Responsive

- Web : priorité.
- Desktop : application locale ou web.
- Mobile : consultation et simulation rapide.

6.4. Accessibilité

- Proposition : niveau AA.
- Contrastes, tailles de police, navigation clavier.

6.5. Design system

- Composants réutilisables : cartes, tableaux, badges, filtres, modales.
- États : vide, chargement, erreur, succès, sélection, désactivé.
- Palette et typographie à définir à partir du logo existant.

---

7. Architecture technique

7.1. Stack proposée

Couche	Technologie
Moteur de simulation	Python
NLG	Python
API	FastAPI ou Flask
Front-end	React / TypeScript
Base de données	SQLite (à terme), Excel (source)
Déploiement	Local, puis serveur ou cloud

7.2. Modules

- `scrape_fminside_attributes.py` : collecte des attributs.
- `compute_fm_rating.py` : calcul de la note FM.
- `match_engine.py` : simulation de match.

- `nlg_engine.py` : génération de texte.
- `api.py` : exposition des données.
- `frontend/` : interface utilisateur.

7.3. Tests

- Tests unitaires sur le scoring et le NLG.
- Tests d'intégration sur le moteur de match.
- Jeux de données de référence.

---

8. Modèle de données

8.1. Base joueurs

- `ID` , `Prénom` , `Nom` , `Nationalité` , `Âge` , `Poste` , `Club` , `Championnat` , `Valeur` , `Taille` , `Statut` , `Catégorie de rôle` , `Pied` , `Moyenne joueur` .
- 47 attributs FM26.
- `FMInside URL` .
- `Note FM poste` , `Note FM rôle` , `Rôle FM retenu` , `Note FM` .

8.2. Base NLG

- Table `scenarios` .
- Table `variantes` .
- Table `phrases` .
- Table `conditions` .
- Table `historique_usage` .
- Table `cooldowns` .

8.3. Base match

- Table `matchs` .
- Table `equipes` .
- Table `joueurs_match` .
- Table `evenements` .
- Table `statistiques` .

---

9. Roadmap et priorités

9.1. Trois priorités pour les 3 prochains mois

1. Finir le scraping des 7 064 joueurs restants.

2. Calculer la note FM sur les 504 joueurs déjà remplis, puis sur toute la base.
3. Coder le moteur NLG et le moteur de simulation.

9.2. Jalons

Échéance	Jalon
Mois 1	Scraping terminé, note FM calculée
Mois 2	Moteur NLG opérationnel
Mois 3	Moteur de match et interface de simulation de saison

9.3. Budget temps

- 5 à 10 h par semaine.

9.4. Équipe

- Seul, avec possibilité de collaboration ponctuelle.

---

10. Métriques de succès

- Nombre de matchs simulés.
- Taux de répétition des phrases.
- Corrélation note FM / résultats.
- Temps passé sur l'application.
- Retours des proches.

---

11. Risques et contraintes

11.1. Risques

- Blocage Cloudflare.
- Changement de structure fminside.
- Données incomplètes.
- Temps insuffisant.
- Perte de motivation.

11.2. Contraintes légales

- Droit d'auteur sur les données FM.
- Droit d'auteur sur les phrases.
- RGPD (si partage).

11.3. Contraintes techniques

- Pas de serveur dédié.
- Pas de budget.
- Temps limité.

---

12. Décisions à prendre

#	Question	Recommandation
1	Accessibilité	Niveau AA
2	Stack front	React + TypeScript
3	Base de données	SQLite à terme
4	API	FastAPI
5	Versioning des données	Oui
6	Tests automatisés	Oui, unitaires et intégration
7	Format PRD	Markdown / HTML
8	Longueur PRD	Moyen (10-20 pages)
9	Moteur : mode rapide / détaillé	Les deux
10	Gestion des tactiques	Oui
11	Gestion des transferts	À terme
12	Gestion des compétitions	Oui

---

13. Annexes

13.1. Documents existants

- Notice technique FM26.
- Script `scrape_fminside_attributes.py`.
- Base `joueurs.xlsx`.
- Rapport `fminside_matching_report.csv`.

13.2. Commandes usuelles

```
 pip install openpyxl beautifulsoup4 curl_cffi
python scripts/scrape_fminside_attributes.py --limit 500
python scripts/compute_fm_rating.py
```

13.3. Arborescence

```
simulafoot/
├── data/
│   ├── joueurs.xlsx
│   ├── joueurs_backup_*.xlsx
│   ├── fminside_matching_report.csv
│   └── .fminside_cache.json
├── scripts/
│   ├── scrape_fminside_attributes.py
│   └── compute_fm_rating.py
├── frontend/
├── api/
└── PROJET_SIMULAFOOT.md
```

Fin du PRD – Version 1.0